

Pipeline de datos para un producto analítico

Reconstruir un ranking exige procedencia, transformaciones y pruebas explícitas.

PROCEDENCIA

de dónde viene cada entrada

TRANSFORMACIÓN

cómo cambia la granularidad

PRUEBAS

qué permite defender el producto

Una tabla correcta todavía puede ser imposible de reconstruir

¿Puede un compañero reconstruir nuestro ranking si solo recibe la tabla final?

RESULTADO RECIBIDO

M11 · Pipeline de datos analítico reproducible

Versión: train-holdout-191d62fc6cc9

Configuración	Media validation/accuracy	Semillas	Puesto
cfg_lr010	0.603515625	2	1
cfg_lr005	0.595703125	2	2

ALCANCE DEL RESULTADO

Resultado docente sin superioridad general



RECONSTRUIR EL RESULTADO

ENTRADAS

¿Con qué datos?

REGLAS

¿Con qué transformaciones?

COMPROBACIONES

¿Bajo qué condiciones?

Entradas y reglas por entregar



Reconstrucción pendiente

Resultado disponible.

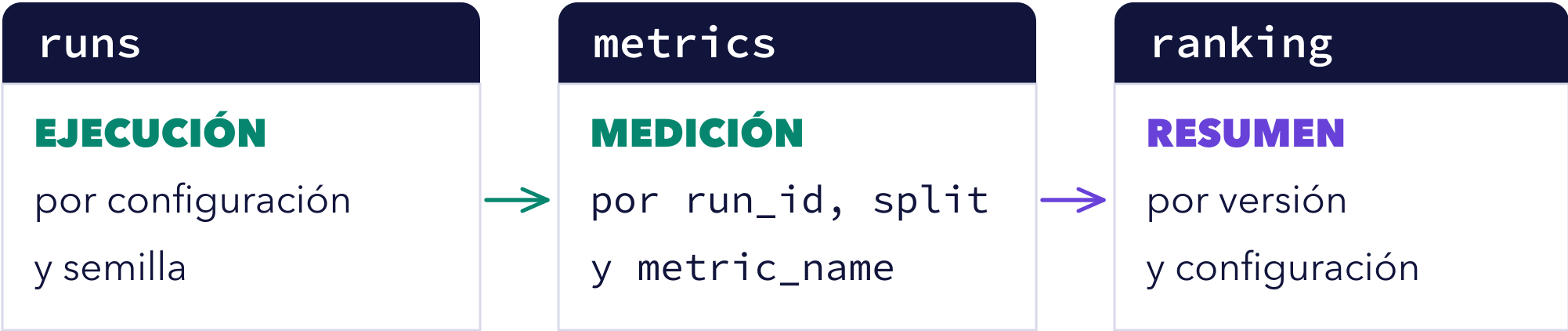
¿Qué falta para que otro compañero reconstruya esta tabla?

Reconstrucción pendiente.

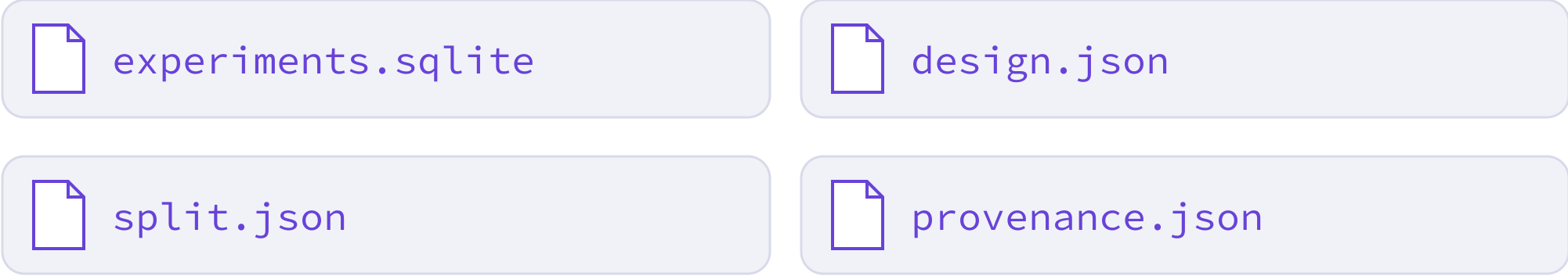
Reconstruir exige recuperar la fila, las entradas y la regla

¿Qué información falta detrás de esas dos filas?

TRES RELACIONES • TRES UNIDADES DE FILA



ENTRADAS QUE DEBEMOS CONSERVAR



¿QUÉ SIGNIFICA UNA FILA AUSENTE?

Fila de metrics ausente



Cada fila tiene una unidad.

¿Qué entrada y qué regla no revela la tabla final?

NULL no equivale a cero.

La comparación conserva población, cobertura y empates

¿Qué contrato deben satisfacer las filas antes de compararlas?

POBLACIÓN Y DISEÑO ESPERADO

`completed` y `validation/accuracy`

Versión: `train-holdout-191d62fc6cc9`

CONFIGURACIÓN × SEMILLA

Configuración	Semilla 11	Semilla 22
cfg_lr005	1 elegible	1 elegible
cfg_lr010	1 elegible	1 elegible

Una ejecución elegible por combinación esperada

CONDICIONES PARA COMPARAR

01 COBERTURA COMPLETA

Antes de la media
entre semillas



02 RANK POR VERSIÓN

Conserva los empates



03 ORDEN DE PRESENTACIÓN

`config_id` solo ordena
la presentación final

Cobertura antes de resumir.

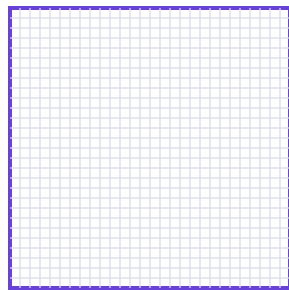
¿De dónde sale el diseño esperado de la comparación?

RANK **conserva los empates.**

El origen oficial no sustituye al holdout local

¿Los tamaños del dataset oficial describen nuestro experimento?

ORIGEN OFICIAL • FASHION-MNIST



28 × 28 píxeles

Dimensiones de imagen

TRAIN OFICIAL

60000

TEST OFICIAL

10000

10 clases

dataset_origin_map

dataset_id	dataset_key
fashion_holdout	fashion_mnist

El mapeo documenta derivación;
los conjuntos no son iguales.

EXPERIMENTO LOCAL

TRAIN / VALIDATION

train

2048

validation

512

Entrenamiento: 2 épocas

VERSIÓN LOCAL

train-holdout-191d62fc6cc9

DERIVACIÓN DOCUMENTADA

Conjuntos distintos del origen

El origen identifica la fuente.

¿El train/test oficial sustituye al train/validation local?

El mapeo documenta la derivación.

Cada frontera del flujo necesita un contrato

¿Cómo se conectan origen, registro y producto sin confundir sus responsabilidades?

DOS ENTRADAS, UNA INTEGRACIÓN

METADATOS DE ORIGEN



REGISTRO EXPERIMENTAL



DISEÑO ESPERADO → COMPROBACIONES

Cobertura y claves se contrastan con el diseño.

CONTRATO EN CADA FRONTERA

COLUMNAS Y TIPOS

¿Qué campos entran y cómo se leen?

CLAVE

¿Cómo se identifica cada entidad?

SIGNIFICADO DE FILA

¿Qué representa cada registro?

ORGANIZACIÓN POSTERIOR

dbt organiza la misma SQL.

No adquiere el HTML ni sustituye los contratos del flujo.

Las flechas declaran dependencias.

¿Qué contrato conecta los metadatos con la integración?

Los contratos delimitan cada paso.

El HTML de clase es un fixture propio con procedencia

¿Qué entrada concreta alimenta la extracción sin depender de la red?

HTML **DOCENTE PROPIO**

references/fixtures/
fashion_metadata.html

`table id="dataset-metadata"`

Lectura por campo de una sola ficha

Campo HTML	Valor de la ficha
dataset_key	fashion_mnist
image_shape	28×28
n_classes	10
official_train	60000
official_test	10000

Ejecución local: el parser lee este archivo.

PROCEDENCIA DEL ARCHIVO

AUTORÍA

course_production

HECHOS CONTRASTADOS

2026-09-03

FUENTE FACTUAL OFICIAL

github.com/
zalando-research/fashion-mnist

SHA-256 · abreviado
df259e06b7f8...

HTTP SERÍA ADQUISICIÓN PREVIA

Este HTML no es una captura web.

Entrada local identificable.

¿Qué evidencia de procedencia acompaña al archivo local?

Procedencia explícita.

El parser identifica tabla, filas y celdas

¿Cómo extraemos la estructura antes de transformar su texto?

DE LA ESTRUCTURA AL TEXTO



HTML simple: sin anidación ni celdas combinadas.

SELECCIÓN Y RECHAZO

Expresiones seleccionadas del parser

```
(dict(attrs).get("id") ==  
"dataset-metadata")
```

```
parser.rows
```

SI SE CUMPLE, SE RECHAZA

```
(parser.table_count != 1  
or len(parser.rows) != 2)
```

```
headers != expected
```

```
len(cells) != len(headers)
```

Las guardas lanzan `ValueError`.

El parser recupera estructura.

¿Quién reconoce la estructura de esta tabla?

El esquema decide si se acepta.

Regex limpia texto y comprueba un patrón acotado

¿Qué puede comprobarse una vez extraído el texto de las celdas?

DEL TEXTO A LA COINCIDENCIA

1 • NORMALIZAR ESPACIOS

" 28 × 28 "

"28 × 28"

2 • COMPARAR EL TEXTO ENTERO

28 × 28

ACEPTADO

La forma textual coincide.

28×28px

RECHAZADO

El sufijo queda fuera.

Coincidir no prueba que 28×28 sea correcto.
El tamaño se contrasta con la fuente factual.

LIMPIEZA Y PATRÓN COMPLETO

LIMPIEZA

```
re.sub(r"\s+", " ", text).strip()
```

Agrupar espacios y recortar extremos.

COINCIDENCIA COMPLETA

```
re.fullmatch(
    r"[0-9]+\s*\s*[0-9]+",
    records[0]["image_shape"]
)
```

Fragmento de la comprobación docente.

seed se lee de su columna;
no se deduce desde run_id.

Regex comprueba la forma.

¿Cumplir el patrón demuestra que el tamaño es correcto?

La fuente confirma el dato.

Una cadena numérica todavía no es un entero

¿Qué contrato tabular falta después de limpiar el texto?

DE TEXTO A COLUMNAS TIPADAS

ANTES • CADENAS

official_train
"60000"

image_shape
"28x28"

DESPUÉS • UNA FILA POR ORIGEN

Columna	Tipo	Valor
dataset_key	String	fashion_mnist
image_height	Int64	28
image_width	Int64	28
n_classes	Int64	10
official_train	Int64	60000
official_test	Int64	10000

La fila sigue identificando el mismo origen: dataset_key.

EXPRESIONES DE POLARS

Extracto de with_columns

```
pl.col("official_train").cast(
    pl.Int64, strict=True
)
```

EXTRAER Y CONVERTIR

```
pl.col("image_shape").str.extract(
    r"^([0-9]+)\s*\s*([0-9]+)$", 1
).cast(pl.Int64).alias(
    "image_height"
)
```

Grupo 1: alto; grupo 2: ancho.

select elige las columnas finales.

Cambian columnas y tipos.

¿Cambió la entidad que representa una fila?

Se conserva el origen de cada fila.

Conversión estricta, NULL, dominio y clave son pruebas diferentes

¿Una conversión estricta garantiza que toda la tabla sea válida?

CUATRO CONTROLES INDEPENDIENTES

Control	Defecto que detecta
<code>cast(..., strict=True)</code> Representación convertible	Conversión inválida Texto que no representa un entero.
<code>null_count()</code> En cada campo requerido	NULL preexistente La conversión no lo elimina.
<code>n_unique(dataset_key)</code> = height: condición esperada	Claves duplicadas Después de controlar NULL.
Positivos y clave no vacía Dimensiones, clases y tamaños	Fuera del dominio Cero, negativos o clave vacía.

EL CASO QUE NO RESUELVE EL CAST

EJEMPLO DIDÁCTICO • NULL PREVIO

```
pl.Series([None, "60000"],
          dtype=pl.String
).cast(pl.Int64, strict=True)
```



[null, 60000]

El cast termina; el NULL permanece.
Falta comprobar los campos requeridos.

No sustituir una medición
ausente por cero.

Convertir no basta.

¿strict=True elimina los NULL que ya estaban presentes?

Ausencia, clave y dominio se validan.

Normalizar metadatos conserva una sola lógica de ranking

¿Hasta dónde debe transformar Polars en este caso?

ENTREGA TIPADA, RESPONSABILIDAD DELIMITADA

POLARS

metadata_frame

Una fila por
dataset_key

Tipado y claves comprobados

Arrow

to_arrow()

DUCKDB · SQL

Población

Resumen

Ranking

UNA SOLA DEFINICIÓN DE RANKING

Aquí Polars normaliza los metadatos;
la lógica analítica se conserva en SQL.

EXTENSIÓN · EAGER Y LAZY

EAGER

Obtiene resultados al operar.

LAZY

Declara un plan.

collect() solicita ejecución.

No implica aceleración demostrada.

Polars **prepara los metadatos.**

¿Por qué conservar el ranking en una sola implementación?

SQL conserva una sola regla.

Extraer en solo lectura y cargar tipos explícitos protege la frontera

¿Cómo llega esa tabla y el registro a una conexión SQL explícita?

EXTRACCIÓN Y CARGA CON RUTAS EXPLÍCITAS

SQLite • mode=ro

experiments.sqlite datasets • configs • runs • metrics

Registro M10: lectura sin modificar la fuente.



Arrow • esquemas explícitos

Incluidos los campos anulables: no inferir solo desde NULL.



DuckDB local • relaciones tipadas

Base de salida separada; tipos, claves y referencias revalidados.

Entrada y salida: rutas explícitas y responsabilidades distintas.

EL DESTINO TAMBIÉN SE COMPRUEBA

TIPO EXPLÍCITO AUN CON NULL

finished_at

failure_reason

LAS RESTRICCIONES NO SE COPIAN

Revalidar claves y referencias.

Comparar el esquema con el contrato.

DISEÑO ESPERADO INDEPENDIENTE

design.json

2 configuraciones × 2 semillas

No se infiere de las filas que sobrevivieron al proceso.

Solo lectura en el origen.

¿Exportar SQLite conserva automáticamente sus restricciones?

Tipos y restricciones se revalidan.

Registrar un objeto y persistir una tabla son operaciones distintas

¿Qué hace persistente una entrada registrada en DuckDB?

TRES OPERACIONES, TRES EFECTOS

1

REGISTRAR

`metadata_input`

Nombre consultable en esta conexión.



2

CREAR LA TABLA

`raw.dataset_metadata`

Contenido guardado en la base de salida.



3

RETIRAR EL NOMBRE

`metadata_input`

La tabla creada sigue disponible.

EL PASO QUE PERSISTE EL CONTENIDO

PYTHON • NOMBRE DE ENTRADA

```
con.register("metadata_input",  
metadata_frame.to_arrow())
```

SQL • BASE LOCAL EN DISCO

```
CREATE OR REPLACE TABLE  
raw.dataset_metadata  
AS SELECT * FROM metadata_input
```

PYTHON • LIBERAR EL NOMBRE

```
con.unregister("metadata_input")
```

Cerrar la conexión tras las cargas.

Al reabrir la base, la tabla permanece.

Registrar hace visible.

¿Qué paso conserva los datos en la base de salida?

CTAS conserva los datos.

Las claves deben conservar cuatro ejecuciones elegibles

¿Qué relaciones hacen que una ejecución entre una sola vez en la población?

SQL • FRAGMENTOS SELECCIONADOS

DISEÑO ESPERADO • ANTES DE CONTAR

```
JOIN raw.expected_design AS e
ON e.dataset_id = r.dataset_id
AND e.dataset_version = d.version
AND e.config_id = r.config_id
AND e.seed = r.seed
```

MÉTRICA OBJETIVO

```
JOIN raw.metrics AS m
ON m.run_id = r.run_id
AND m.split = 'validation'
AND m.metric_name = 'accuracy'

Después de los JOIN de mapeo y metadatos:
WHERE r.status = 'completed'
```

GRAFO DE CLAVES Y CARDINALIDAD

runs → datasets / configs
dataset_id / config_id

dataset_origin_map
dataset_id → dataset_key



dataset_metadata
JOIN por dataset_key único

4 esperadas → 4 elegibles

Si se cumplen claves y referencias.

Duplicados multiplican; ausencias eliminan.

Una fila por ejecución elegible.

¿Qué podría multiplicar o eliminar las cuatro filas esperadas?

Las claves sostienen el conteo.

Cobertura completa permite resumir; RANK conserva la comparación

¿Cómo pasamos de cuatro ejecuciones a dos configuraciones comparables?

COBERTURA COMPLETA

```
HAVING COUNT(*) = e.expected_seed_count
AND COUNT(DISTINCT s.seed) = e.expected_seed_count
```

PRECONDICIÓN
Diseño esperado
único y no vacío

VENTANA DE COMPARACIÓN

```
RANK() OVER (
  PARTITION BY dataset_version
  ORDER BY mean_validation_accuracy DESC
) AS rank_in_version
```

RESULTADO • MISMA VERSIÓN
train-holdout-191d62fc6cc9
Versión inequívoca en este caso

Configuración	Media	Semillas	Puesto
cfg_lr010	0.603515625	2	1
cfg_lr005	0.595703125	2	2

COBERTURA → RESUMEN → COMPARACIÓN
¿Dos semillas cualesquiera bastan? ¿config_id rompe el empate?

Duplicar metadatos puede multiplicar filas sin nuevas corridas

¿Qué falla si una ficha de metadatos aparece dos veces?

COPIA DIDÁCTICA • CLAVE DUPLICADA

Dos copias de la misma ficha: fashion_mnist.

run_id		run_id	ficha
run_1		run_1	A
		run_1	B
run_2		run_2	A
		run_2	B
run_3		run_3	A
		run_3	B
run_4		run_4	A
		run_4	B

4 ejecuciones × 2 fichas = 8 filas integradas

CONTAR FILAS NO ES CONTAR CORRIDAS

DIAGNÓSTICO DEL JOIN

COUNT (*)	8
COUNT(DISTINCT run_id)	4

UNA MEDIA IGUAL NO LO DESCARTA

Duplicar simétricamente puede conservar la misma media.

El resumen contractual rechaza la multiplicidad antes del ranking.

A y B rotulan copias de la ficha;
run_1...run_4 son rótulos didácticos.

Más filas no son más corridas.

¿Una media igual descarta la duplicación?

Una media igual no basta.

La reparación recupera la clave de la fuente

¿Dónde debemos detener y corregir esa multiplicación?

REPARACIÓN EN LA COPIA DOCENTE

1

DETECTAR LA CLAVE DUPLICADA

`dataset_metadata.dataset_key`



2

RECHAZAR LA FUENTE

Detener la integración antes del JOIN.



3

RESTAURAR LA FICHA VERIFICADA

Investigar la causa; corregir solo la copia.



4

REPETIR LA INTEGRACIÓN

Comprobar una fila por ejecución elegible.

COMPROBAR ANTES Y DESPUÉS

ANTES DEL JOIN • METADATOS

Campos requeridos sin NULL.

`n_unique(dataset_key) = height`

Condición esperada de unicidad.

DESPUÉS DEL JOIN • EJECUCIONES

`run_id` único

4 filas; 4 identificadores distintos.

Para el diseño esperado de este caso.

LA CAUSA DECIDE LA REPARACIÓN

DISTINCT y borrar arbitrariamente
no justifican qué ficha conservar.

Corregir la fuente con evidencia.

¿Qué etapa corriges y qué comprobación repites?

Repetir claves y cardinalidad.

Un hash y una prueba responden preguntas diferentes

¿Qué evidencia distingue conservar una entrada de validar sus datos?

UNA ENTRADA, DOS COMPROBACIONES

COPIA CONGELADA • CON DUPLICADO

dataset_key
fashion_mnist
fashion_mnist



HASH

¿Cambió el archivo?

Hash coincide

Bytes conservados.

UNICIDAD

¿Se repite la clave?

La prueba falla

1 clave distinta; 2 filas.

La misma copia alimenta las dos comprobaciones.

CONSERVAR NO IMPLICA VALIDAR

REFERENCIA DEL EJEMPLO

Se congeló una copia que ya contenía la ficha duplicada.

PREGUNTAS DIFERENTES

Hash: identidad de los bytes.

Prueba: predicado sobre datos.

EL MISMO ERROR PUEDE REPETIRSE

Una entrada idéntica puede reproducir el mismo duplicado.

El hash detecta cambios de bytes.

Explica la diferencia usando la ficha duplicada.

La prueba evalúa un predicado.

La falla de una entrada viaja por sus dependencias

¿Qué depende de los metadatos y cómo puede propagarse una falla al ranking?

RECUPERAMOS EL FLUJO DE LA SESIÓN 1

1

METADATOS

Copia didáctica: clave duplicada.



2

INTEGRACIÓN

La población necesita esos metadatos.



3

RESUMEN

Necesita la población integrada.



4

RANKING

Necesita el resumen.

Flechas: dependencia de datos.

LOCALIZAR EL EFECTO DE UNA FALLA

RECHAZAR ANTES DE INTEGRAR

La clave duplicada debe detenerse en la entrada de metadatos.

SI SE OMITE ESE CONTROL

El JOIN puede multiplicar filas.

El resumen debe controlar cobertura y multiplicidad.

Las flechas señalan qué producto necesita los datos de otro.

No representan un nuevo build.

La integración depende de la fuente.

¿Qué resultados dependen de la ficha de metadatos?

El ranking depende del resumen.

El orden válido nace de las dependencias

¿Cómo evitamos depender del orden histórico de las celdas?

ORDEN DE CONSTRUCCIÓN DESDE CERO

1

`stg_validation_runs`

Produce la población elegible.



2

`int_config_summary`

Necesita staging para resumir.



3

`mart_config_ranking`

Necesita el resumen para comparar.

Cada entrada debe estar disponible antes de consumirla.

EL ARCHIVO NO CREA SU ENTRADA

INTENTO: RESUMEN PRIMERO

`int_config_summary`

requiere `stg_validation_runs`

ENTRADA TODAVÍA NO DISPONIBLE

La construcción limpia no puede avanzar.

EL ORDEN RESPETA EL GRAFO

Escribir primero un archivo
no hace disponible su entrada.

Este grafo es acíclico.

Primero, la relación necesaria.

¿Puede construirse el resumen sin una población elegible?

Después, quien la consume.

Un modelo SQL transforma una relación; no entrena un clasificador

¿Qué unidad mantenible representa cada transformación SQL?

MODELO DE MACHINE LEARNING

Clasificador del experimento

Produce predicciones.

MODELO SQL EN dbt

Transformación relacional

Produce una relación.

TRES TRANSFORMACIONES DE ESTE CASO

Fuentes



Staging



Resumen



Mart

Staging integra claves, diseño y métrica.

El resumen controla cobertura y calcula la media.

El mart prepara la comparación.

INGENIERÍA ANALÍTICA

Transformaciones versionadas,
definiciones y pruebas.

Staging integrado: decisión docente
de este caso; no arquitectura universal.

Aquí dbt no reentrena el clasificador.

Un clasificador produce predicciones.

¿Un modelo dbt entrena el clasificador?

Un modelo SQL produce una relación.

Cada uno de los tres modelos conserva un contrato de fila

¿Qué significa una fila en cada modelo antes de construirlo?

TRES MODELOS • TRES CONTRATOS

Modelo SQL	Una fila representa	Responsabilidad
stg_validation_runs	Ejecución elegible	Claves + diseño + métrica
int_config_summary	Versión/config completa	Cobertura + media
mart_config_ranking	Versión/config con puesto	Comparación

PRECONDICIONES FUERA DEL MART

expected_design existe fuera de las filas sobrevivientes.
El nombre mart no certifica calidad.

DÓNDE CAMBIA EL SIGNIFICADO DE LA FILA

LA AGREGACIÓN CAMBIA LA UNIDAD

4 ejecuciones elegibles



2 versiones/configuraciones

El resumen calcula una media por configuración.

LA VENTANA CONSERVA ESA UNIDAD

2 versiones/configuraciones con puesto

RANK añade una columna; no vuelve a agregar.

Claves y referencias se comprueban antes.

El resumen cambia la granularidad.

¿En cuál modelo cambia el significado de una fila?

RANK solo añade el puesto.

source nombra la entrada que ya preparamos

¿Cómo usa dbt las fuentes físicas que consultamos el día1?

PLANTILLA dbt

source IDENTIFICA UNA FUENTE DECLARADA

```
FROM {{ source('raw', 'runs') }} AS r
JOIN {{ source('raw', 'datasets') }} AS d
ON d.dataset_id = r.dataset_id
```



dbt **resuelve la plantilla antes de enviarla.**

RESPONSABILIDAD DE source

Nombra una entrada ya preparada.

No descarga, crea ni valida automáticamente los datos.

SQL QUE RECIBE DUCKDB

RELACIONES FÍSICAS RESUELTAS

```
FROM raw.runs AS r
JOIN raw.datasets AS d
ON d.dataset_id = r.dataset_id
```

LA ENTRADA YA FUE CARGADA

raw.runs existe en la base local.

DuckDB ejecuta la SQL compilada.

DuckDB no recibe Jinja sin resolver.

source identifica la entrada.

¿source descarga datos o identifica una relación preparada?

La relación física ya fue cargada.

ref declara una dependencia; SQL escribe el JOIN

¿Cómo se conecta un modelo con el resultado de otro?

FRAGMENTO DEL RESUMEN

```
FROM {{ ref('stg_validation_runs') }} AS s
JOIN expected_counts AS e
ON e.dataset_id = s.dataset_id
AND e.dataset_version = s.dataset_version
AND e.config_id = s.config_id
```

ref

Referencia un modelo.

Declara dependencia.

JOIN / ON

Combina relaciones.

Fija correspondencia.

Las tres condiciones de clave permanecen en SQL.

DOS OPERACIONES DIFERENTES

DEPENDENCIA DEL GRAFO

stg_validation_runs



int_config_summary

ref permite ordenar la construcción.

UNIÓN RELACIONAL

JOIN expected_counts AS e

ON contiene las claves de unión.

ref no ejecuta el JOIN.

ref declara dependencia.

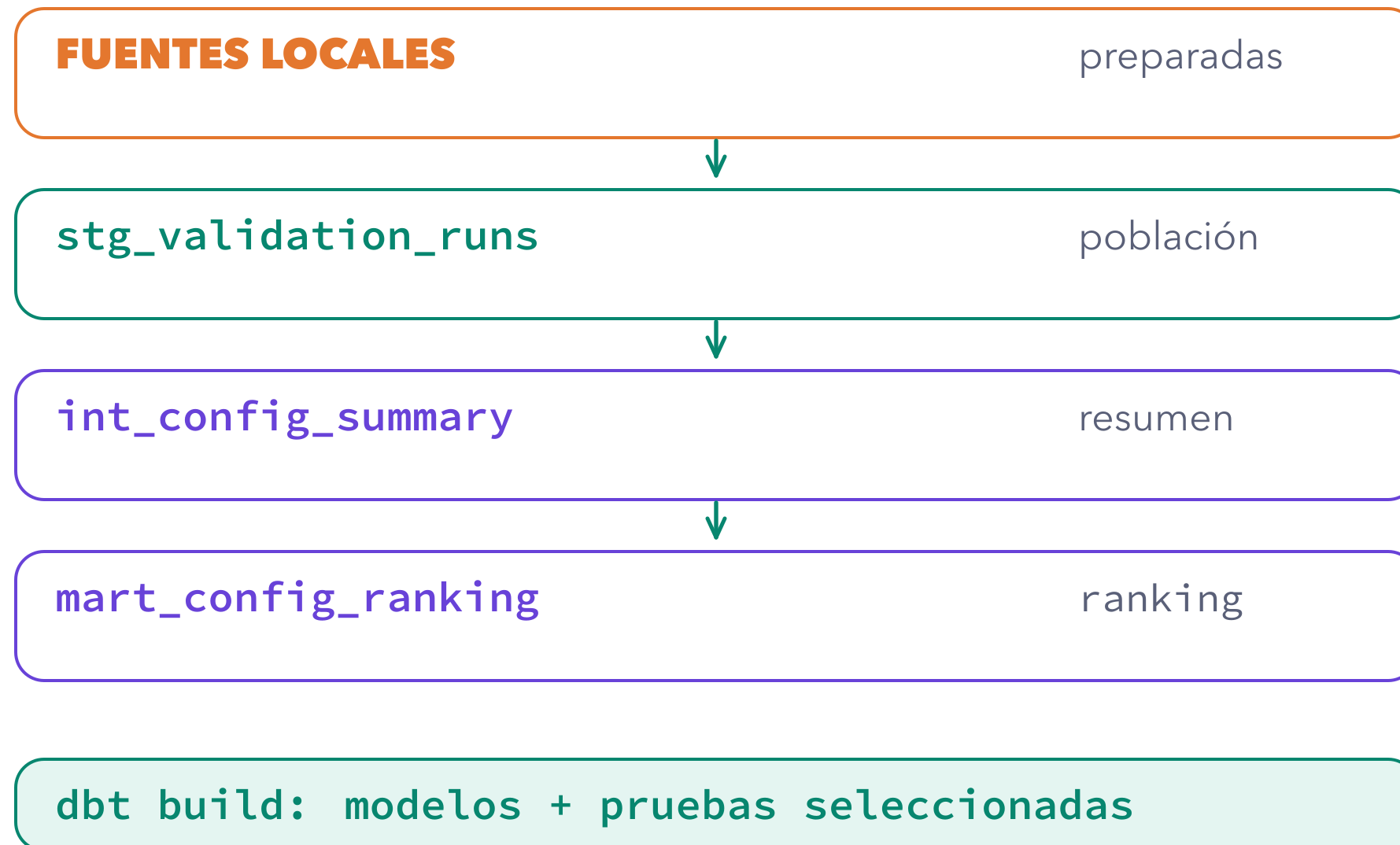
¿Dónde se declara la dependencia y dónde se escribe la unión?

JOIN escribe la unión.

Construir el grafo conserva la SQL conocida

¿En qué orden puede dbt construir los tres modelos?

GRAFO DE CONSTRUCCIÓN



LA MISMA VENTANA DE RANKING

```
RANK() OVER (  
  PARTITION BY dataset_version  
  ORDER BY mean_validation_accuracy DESC  
) AS rank_in_version  
FROM {{ ref('int_config_summary') }}
```

EVIDENCIA DEL CASO VERIFICADO

3 modelos: success
Pruebas seleccionadas: pass
SQL directa = contenido dbt

Resultado local; no garantía universal.

El grafo fija el orden.

¿Qué requiere el ranking en una reconstrucción limpia?

dbt **construye y prueba.**

Una prueba certifica solo el predicado que ejecuta

¿Qué debe verificarse además de construir los modelos?

AUDITORÍA ANTES DEL JOIN

CLAVES • REFERENCIAS • REQUERIDOS



INNER JOIN **puede descartar una fila**

stg_validation_runs

Lo descartado ya no llega a estas pruebas.

PRUEBAS SOBRE STAGING

```
columns:
- name: run_id
  data_tests: [not_null, unique]
- name: validation_accuracy
  data_tests: [not_null]
```

not_null detecta NULL

unique detecta duplicados

Cada prueba certifica solo su propio predicado.

Auditar antes del JOIN.

¿Por qué revisar solo después del INNER JOIN puede ocultar el error?

Cada prueba tiene un alcance.

El dominio de accuracy es una regla explícita

¿Una columna no nula contiene necesariamente una accuracy válida?

PRUEBA SINGULAR • DOMINIO

accuracy_domain.sql

```
SELECT run_id, validation_accuracy
FROM {{ ref('stg_validation_runs') }}
WHERE validation_accuracy IS NULL
OR NOT isfinite(validation_accuracy)
OR validation_accuracy < 0
OR validation_accuracy > 1
```

```
finita AND 0 <= validation_accuracy <= 1
```

CÓMO LEER EL RESULTADO

1+ FILAS

Cada fila devuelta viola la regla de dominio.

0 FILAS

No encontró violaciones de esta regla.

NO DEMUESTRA

cobertura de ejecuciones

Una fila devuelta viola la regla.

¿Cero filas en esta prueba demuestra que el producto está completo?

Cero filas no demuestra cobertura.

La cobertura empieza por lo esperado, no por lo que sobrevivió

¿Cómo detectamos una ejecución que desapareció antes del resumen?

PRUEBA DE COBERTURA • EXTRACTO

expected_coverage.sql

```
FROM {{ source('raw', 'expected_design') }} AS e
LEFT JOIN {{ ref('stg_validation_runs') }} AS s
-- Extracto: se omiten claves ON y GROUP BY
HAVING COUNT(s.run_id) <> 1
```

DISEÑO INDEPENDIENTE • 4 COMBINACIONES

cfg_lr00511

cfg_lr00522

cfg_lr01011

cfg_lr01022

exactamente 1 ejecución por combinación

Un mart vacío puede pasar único y seguir incompleto.

PRUEBA EJECUTADA • metrics VACÍA

metrics: 0 filas

→

staging: 0 filas

uniquePASS

expected_coverageFAIL

int_config_summaryskipped

mart_config_rankingskipped

El diseño conserva lo esperado.

¿Cómo detecta el diseño independiente una combinación ausente?

Un vacío puede pasar único.

La métrica omitida revela la diferencia entre consulta y construcción

¿Qué ocurre si omitimos una fila validation/accuracy en una copia?

ALTERACIÓN CONTROLADA Copia didáctica: se omitió una fila validation/accuracy.

SQL AISLADA • CONSECUENCIA LÓGICA

stg_validation_runs
una ejecución menos

↓

int_config_summary
excluye la configuración que quedó incompleta

Explica qué harían las consultas por separado.

dbt build • RESULTADO VERIFICADO

stg_validation_runs	success
expected_coverage	FAIL
int_config_summary	skipped
mart_config_ranking	skipped

Cobertura falla. **No se construye un ranking nuevo.**

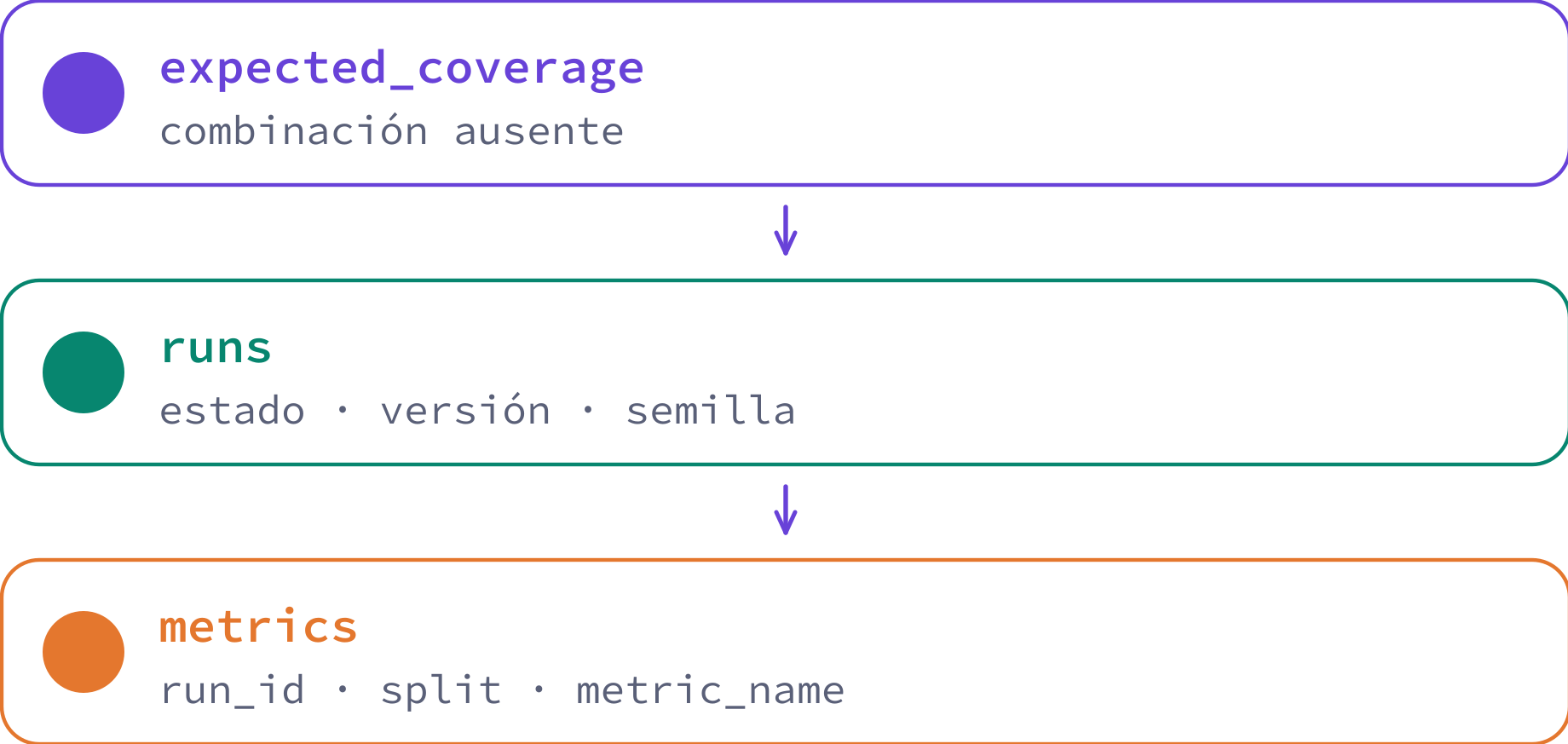
La consulta explica la consecuencia.
¿El build fallido construyó un ranking nuevo con una fila menos?

El build detiene descendientes.

La combinación ausente permite rastrear la causa

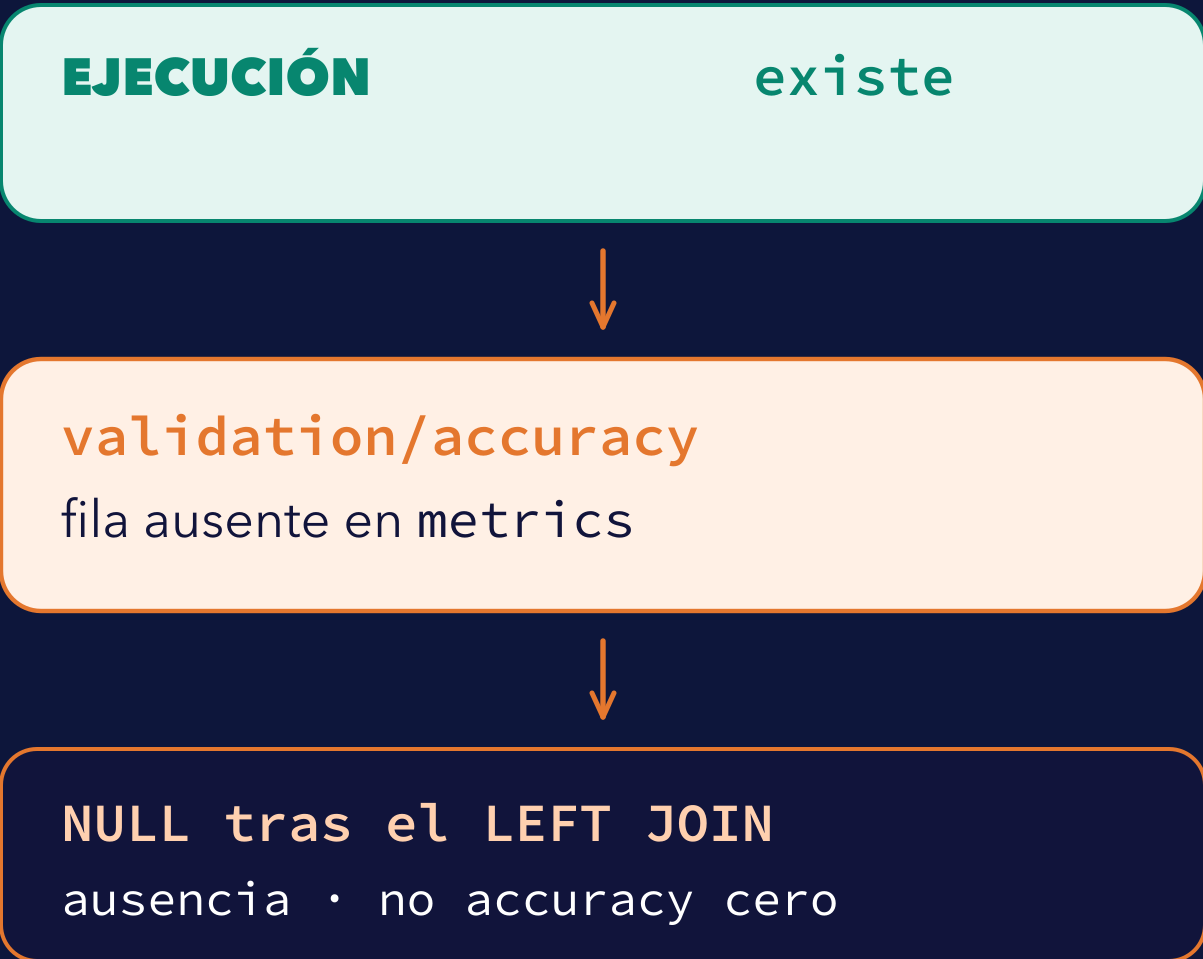
¿Cómo distingue la auditoría ejecución ausente de métrica ausente?

RUTA DE DIAGNÓSTICO



LEFT JOIN **conserva la combinación investigada.**

CAUSA EN LA COPIA DIDÁCTICA



Sin inventar run_id o combinación.

Seguir la combinación, no adivinar.

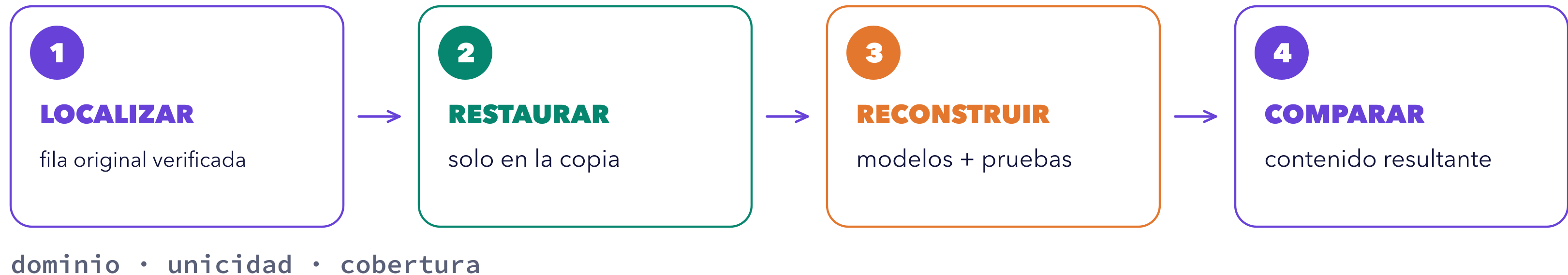
¿Qué tabla inspeccionas si runs existe pero falta validation/accuracy?

NULL aquí significa ausencia.

Corregir restaura evidencia verificada y obliga a reconstruir

¿Qué corrección justifica volver a aceptar el producto?

RECUPERACIÓN BASADA EN EVIDENCIA



LÍMITE HONESTO

Sin evidencia original: declarar el producto incompleto.

rellenar con 0

rellenar con la media

hace pasar una regla; no recupera la medición

Restaurar evidencia, no fabricar.

¿Basta rellenar con cero para que pase `not_null`?

Corregir obliga a reconstruir.

Dos reconstrucciones se comparan por su contenido

¿Qué demuestra repetir el flujo desde salidas independientes?

RECONSTRUCCIÓN • ORDEN COMPLETO

1. Procedencia e integridad
- ↓
2. Extraer en solo lectura
- ↓
3. Normalizar metadatos
- ↓
4. Cargar relaciones tipadas
- ↓
5. Comprobar dominios y claves
- ↓
6. Modelos y pruebas
- ↓
7. Comparar e interpretar

Versión: train-holdout-191d62fc6cc9

Configuración	Media	Semillas	Puesto
cfg_lr010	0.603515625	2	1
cfg_lr005	0.595703125	2	2

mismo esquema • claves • filas • valores

DOS SALIDAS INDEPENDIENTES

MISMO CONTENIDO ORDENADO

two_clean_reconstructions_equal = true
m10_sources_unchanged = true
No se exige igual hash de bases o logs.

Comparar contenido ordenado.

¿Dos bases necesitan el mismo hash para representar la misma respuesta?

Igual respuesta ≠ igual archivo.

Transferir el método exige redefinir el contrato

¿Qué reglas cambian si el próximo problema usa otra métrica?

CONTRATO ACTUAL • accuracy

DOMINIO

$[0, 1]$

ORDEN

mayor es mejor • DESC

IDENTIDAD

versión inequívoca en este caso

REGRESIÓN HIPOTÉTICA

¿qué métrica?

¿minimizar o maximizar?

¿qué dominio?

¿qué diseño?

La regla depende de la métrica.

REDEFINIR EL CONTRATO

entradas • identidad • mapeo • fila • población • métrica • diseño • tests

Si se reutiliza una versión: ampliar clave y partición, por ejemplo con dataset_id.

2048/512 • 2 épocas • 2 semillas → sin superioridad general demostrada

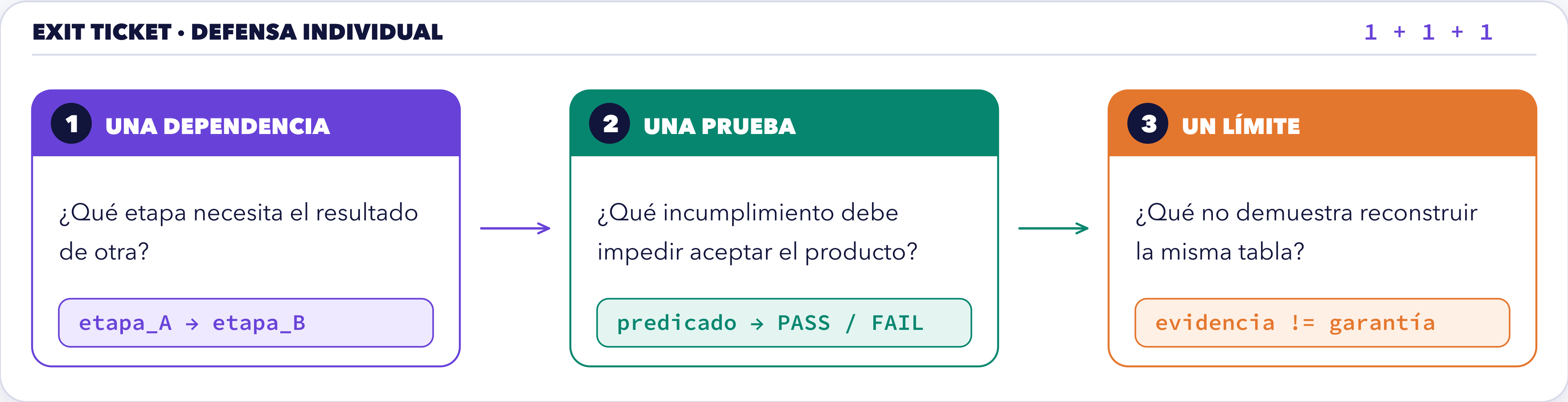
El método se transfiere.

¿Conservarías automáticamente $[0,1]$ y DESC en regresión?

El contrato se redefine.

Defender el producto exige una dependencia, una prueba y un límite

¿Cómo defenderías el producto completo sin prometer más de lo comprobado?



UNA DEFENSA CONECTA LAS TRES DECISIONES

dependencia + prueba + límite

Nombra una de cada una y explica cómo ayuda a reconstruir el producto.

Nombra.

Justifica.

Conecta.

Defiende el producto: una dependencia, una prueba y un límite.